

```

//=====
//
//      FICHE SYNTHESE - LANGAGE C
//
//=====

// Les commentaires en majuscules sont fortement conseillés.
// Les commentaires en minuscules sont des explications pour cette fiche synthèse.

// Commentaire sur une seule ligne.

/*
Commentaire sur
plusieurs lignes.
*/

//=====
//      INCLUSIONS DES HEADERS
//=====

// Un fichier source (*.c) est toujours associé à un header (*.h) du même nom.
// Le header contient la déclaration (nom et type) des fonctions et variables,
// alors que le fichier source contient la description (programme) de celles-ci.
#include <un_autre_fichier.h>

//=====
//      CONSTANTES PREDEFINIES
//=====

// Le compilateur remplacera toutes les constantes par leur valeur.
#define CONSTANCE    10           // Entier
#define PI            3.14159     // Réel
#define CARACTERE     'a'         // Caractère
#define CHAINE        "Coucou"    // Chaîne de caractères

//=====
//      VARIABLES GLOBALES
//=====

// Les variables globales sont visibles et modifiables par la totalité du code.
// Pour cette dernière raison, l'usage de variables globales est déconseillé.
// Elles sont déclarées, voire initialisées, ici.

char variable_globale_1 = 'b';
char variable_globale_2 = 8;

//=====
//      FONCTIONS
//=====

// Une fonction peut avoir plusieurs paramètres d'entrée et un paramètre de sortie.
// Le type de la fonction correspond au type du paramètre de sortie.
short int fonction1(short int entree1, char entree2)
{
    // DECLARATIONS DES VARIABLES LOCALES
    // Ces variables ne sont visibles que par cette fonction
    short int var_locale;

    // CODE DE LA FONCTION
    var_locale = entree1 + entree2;           // Lecture des entrées
    var_locale = var_locale + PI;            // Utilisation de constante
    var_locale = var_locale - variable_globale_2; // Utilisation de var globale

    return var_locale
}

```

```

//=====
//  ROUTINE PRINCIPALE
//=====
// La fonction main() est le point d'entrée du programme
void main()
{
    // DECLARATION DES VARIABLES LOCALES
    // Ces variables ne sont visibles que par cette fonction
    short int variable_1;
    short int i;
    short int tableau[20];          // Déclaration d'un tableau contenant 20 entiers.
                                    // Les cases du tableau sont numérotées de 0 à 19.

    // CODE DE LA FONCTION

    // Structure conditionnelle : SI "condition vraie" FAIRE aaa, SINON FAIRE bbb
    if(variable_1 > CONSTANTE)
    {
        // si condition vraie
    }
    else
    {
        // sinon
    }

    // Boucle for : FAIRE instructions N FOIS
    for(i=0; i < 20 ; i++)
    {
        // Pour i commençant à 0 (i=0), et augmentant par pas de 1 (i++)
        // Exécuter la boucle tant que i < 20

        tableau[i] = 0;          // Ecrit la valeur 0 dans la case i du tableau
    }

    // Boucle while : TANT QUE "condition vraie", FAIRE instructions
    while(variable_1 >= variable_globale_2)
    {
        // Instructions
    }

    // BOUCLE INFINIE
    while(1)
    {
        // Les instructions dans cette boucle seront exécutées à l'infini.
    }
}

//=====
//  Type et représentation des variables
//=====

// ATTENTION : Cette partie ne doit pas faire partie du code
char          caractere;          // entier représenté sur 8 bits ou caractère
short int     entier_court;        // entier représenté sur 16 bits
long int      entier_long;         // entier représenté sur 32 bits
long long int entier_looong;        // entier représenté sur 64 bits
int           entier;              // entier représenté sur le même nombre
                                    // de bits que le processeur (à éviter)

signed short int  entier_signe;    // entier positif ou négatif
unsigned short int entier_positif; // entier uniquement positif

float           reel_court;         // réel représenté sur 32 bits
double          reel_long;          // réel représenté sur 64 bits
                                    // Il est préférable de ne pas utiliser de
                                    // réel dans les micro-contrôleurs et
                                    // processeurs 8-bits ou 16-bits.

const short int  var_constante;    // const : la variable n'est pas modifiable

```